SQL JOIN Types

Explained Visually

This guide explains every type of SQL JOIN with pictures. You will see two circles: A (employees) and B (departments). The colored area shows which rows are included in the result. The gray area shows which rows are NOT included.

We use the Oracle HR database for all examples.

Our Example Tables:

A: employees (3 rows)

employee_id	first_name	department_id
101	Neena	10
102	Lex	NULL
107	Alice	60

B: departments (3 rows)

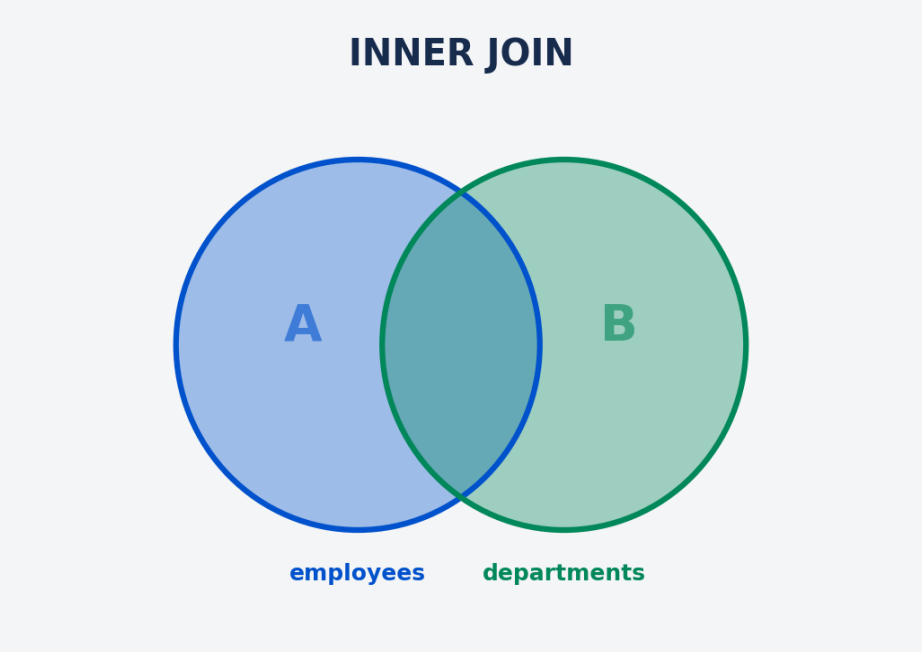
department_id	department_name
10	Administration
60	IT
90	Executive

Match key: department_id. Neena matches dept 10. Alice matches dept 60. Lex has NULL department. Dept 90 (Executive) has no employee.

Keep this in mind: we have 1 match (Neena), 1 match (Alice), 1 no-match from A (Lex), 1 no-match from B (Executive dept).

INNER JOIN

Only rows that exist in BOTH tables



employees = A | departments = B

INNER JOIN returns only the rows that match in BOTH tables. If an employee has no department, or a department has no employee, that row is not included. Look at the picture: only the overlapping area (where A and B meet) is colored. Everything else is gray.

```
SELECT e.first_name, d.department_name
FROM hr.employees e
  INNER JOIN hr.departments d
    ON e.department_id = d.department_id;
```

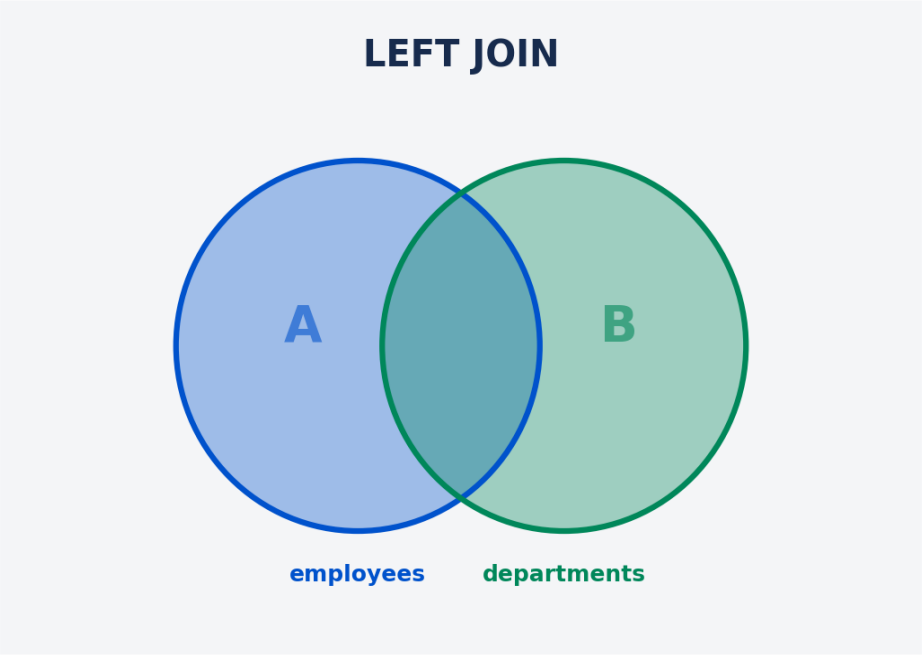
Result (2 rows):

first_name	department_name
Neena	Administration
Alice	IT

Lex is not here (NULL department). Executive dept is not here (no employee). INNER JOIN removed both of them.

LEFT JOIN

All from A + matching from B (NULL if no match)



employees = A | departments = B

LEFT JOIN returns ALL rows from the left table (A), plus matching rows from the right table (B). If there is no match, the B columns get NULL. Look at the picture: the entire A circle is colored (all employees are kept). Lex is still here, but her department is NULL.

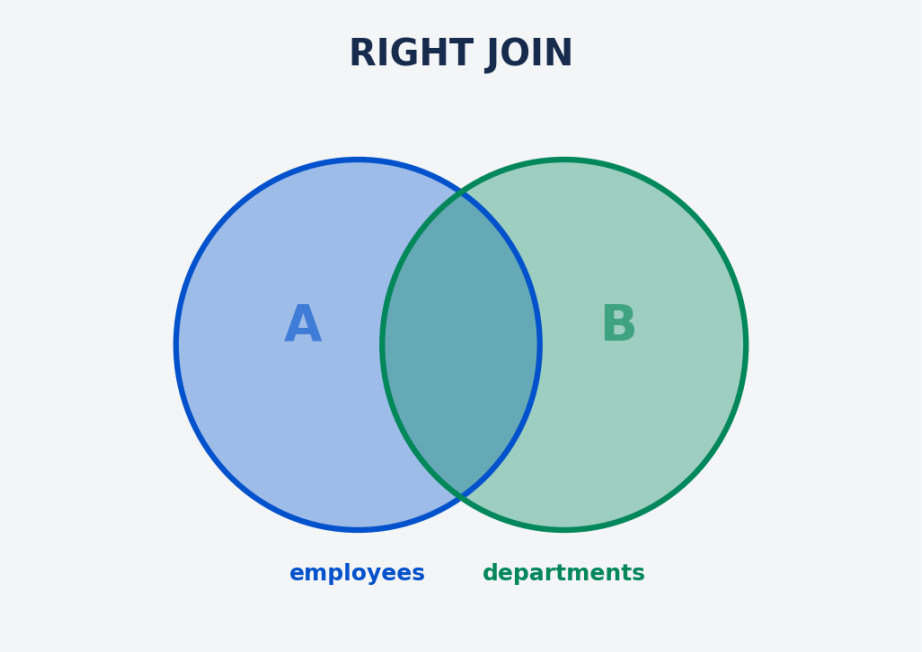
```
SELECT e.first_name, d.department_name
FROM hr.employees e
  LEFT JOIN hr.departments d
    ON e.department_id = d.department_id;
```

first_name	department_name
Neena	Administration
Lex	NULL
Alice	IT

Lex is here with NULL department. Executive dept is NOT here (it is only in B).

RIGHT JOIN

All from B + matching from A (NULL if no match)



employees = A | departments = B

RIGHT JOIN returns ALL rows from the right table (B), plus matching rows from the left table (A). It is the opposite of LEFT JOIN. If a department has no employee, the A columns get NULL. Look at the picture: the entire B circle is colored.

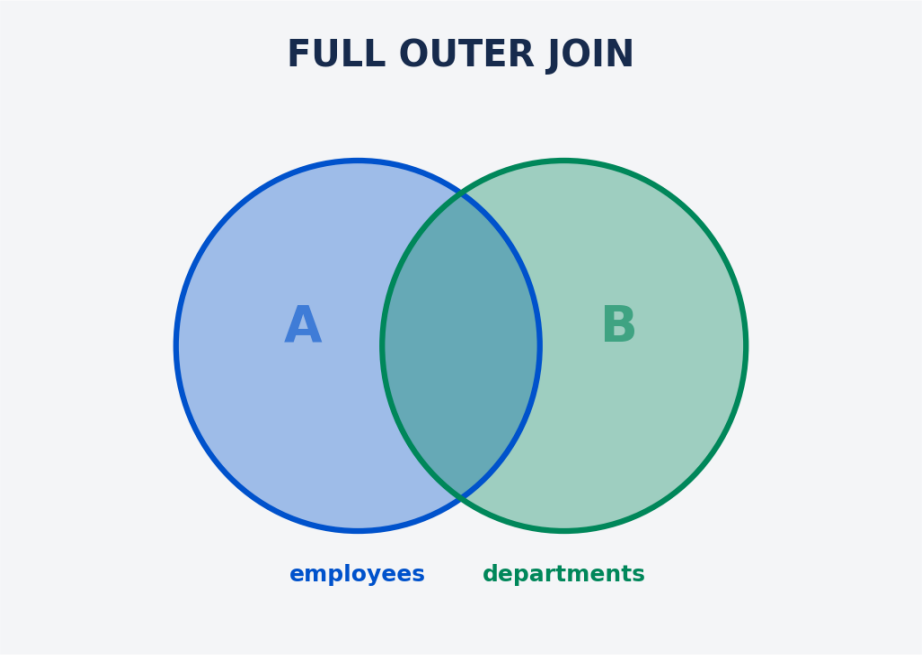
```
SELECT e.first_name, d.department_name
FROM hr.employees e
  RIGHT JOIN hr.departments d
    ON e.department_id = d.department_id;
```

first_name	department_name
Neena	Administration
Alice	IT
NULL	Executive

Executive dept is here with NULL employee. Lex is NOT here (she is only in A).

FULL OUTER JOIN

All from A + All from B (NULL where no match)



employees = A | departments = B

FULL OUTER JOIN returns ALL rows from BOTH tables. If there is no match on either side, the missing side gets NULL. Look at the picture: everything is colored. No rows are lost. This is the most complete JOIN.

```
SELECT e.first_name, d.department_name
FROM hr.employees e
  FULL OUTER JOIN hr.departments d
    ON e.department_id = d.department_id;
```

first_name	department_name
Neena	Administration
Lex	NULL
Alice	IT
NULL	Executive

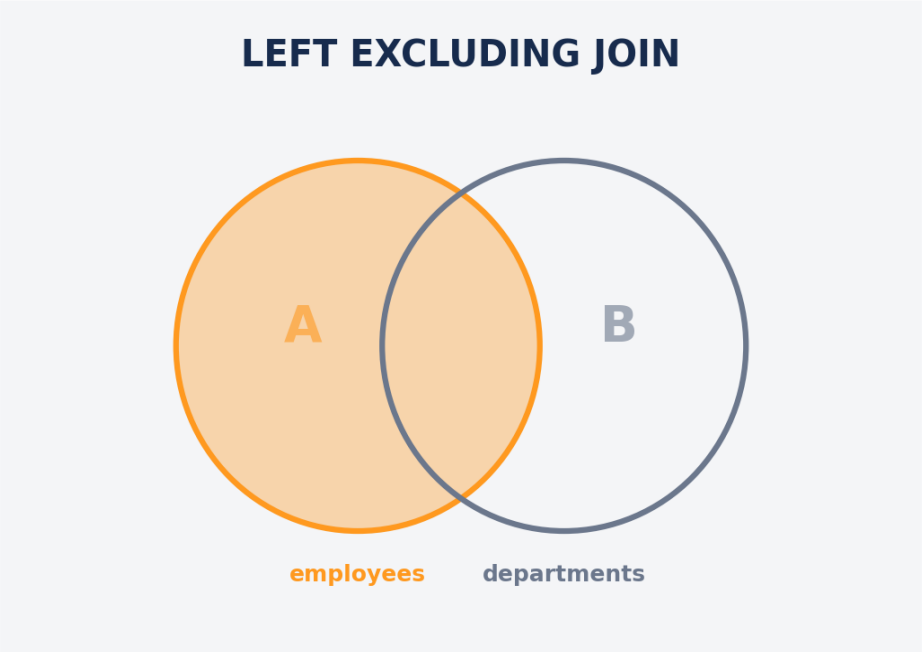
Everyone is here. Lex has NULL department. Executive has NULL employee. Nothing is lost.

Special JOIN Types

These JOINs are less common but still useful. They return only the rows that do NOT match.

LEFT EXCLUDING JOIN

Only A rows with NO match in B



employees = A | departments = B

This returns only rows from A that have NO match in B. It is like LEFT JOIN but without the matching rows. Only the orange part (A only, no overlap) is returned.

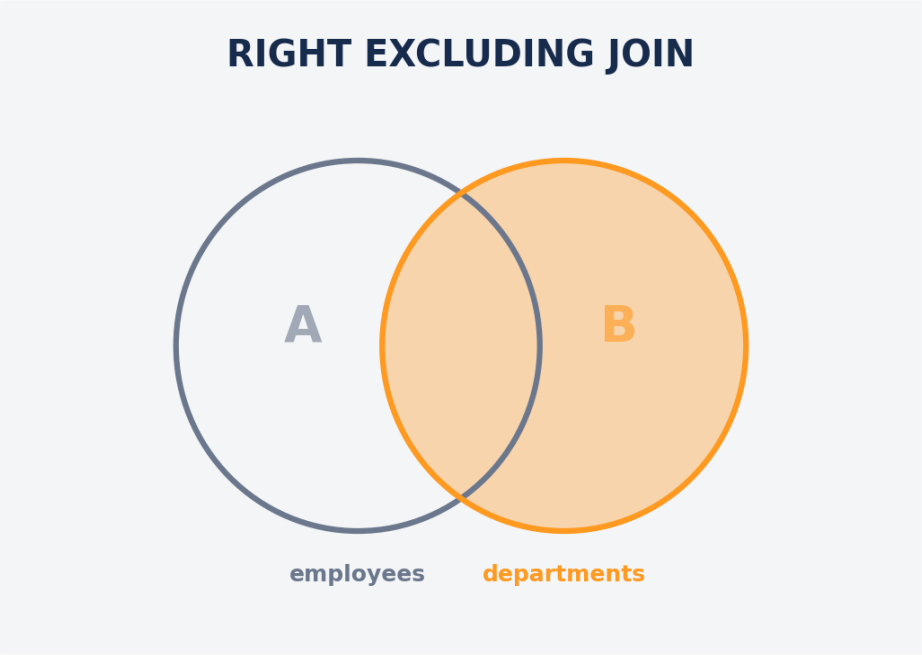
```
SELECT e.first_name, d.department_name
FROM hr.employees e
  LEFT JOIN hr.departments d
    ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

first_name	department_name
Lex	NULL

Only Lex. She has no department.

RIGHT EXCLUDING JOIN

Only B rows with NO match in A



employees = A | departments = B

This returns only rows from B that have NO match in A. Like RIGHT JOIN but without the matching rows.

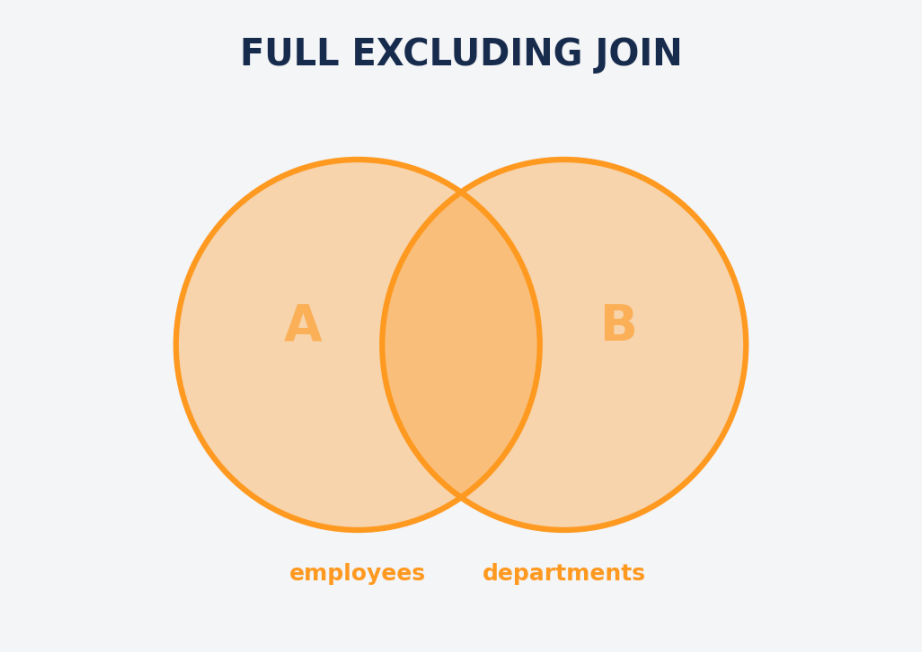
```
SELECT e.first_name, d.department_name
FROM hr.employees e
  RIGHT JOIN hr.departments d
    ON e.department_id = d.department_id
WHERE e.employee_id IS NULL;
```

first_name	department_name
NULL	Executive

Only Executive dept. It has no employee.

FULL EXCLUDING JOIN

All non-matching rows from A and B



employees = A | departments = B

This returns rows from A with no match AND rows from B with no match. The overlapping (matching) rows are NOT included. It is the opposite of INNER JOIN.

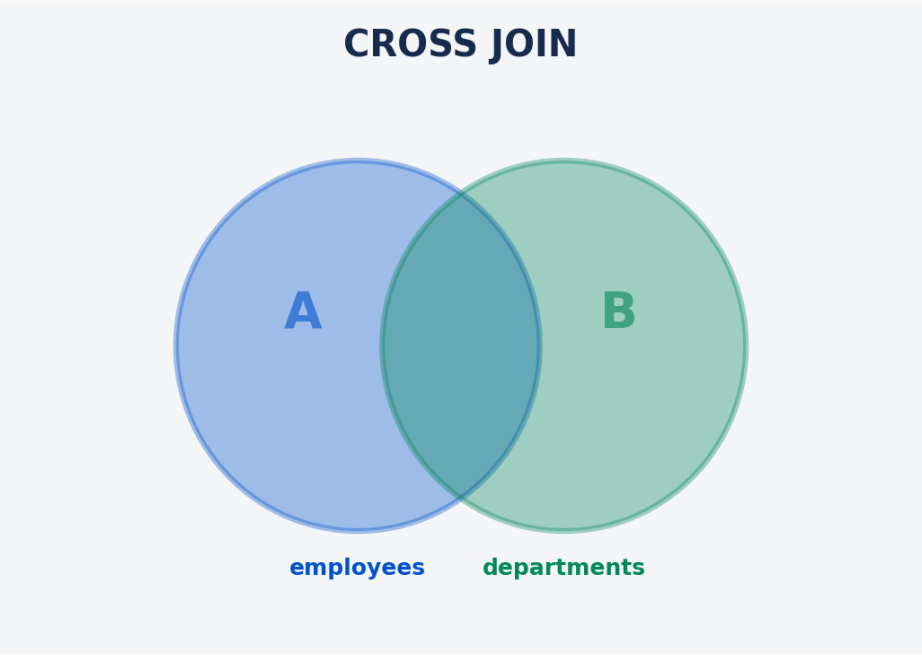
```
SELECT e.first_name, d.department_name
FROM hr.employees e
  FULL OUTER JOIN hr.departments d
    ON e.department_id = d.department_id
WHERE e.employee_id IS NULL
    OR d.department_id IS NULL;
```

first_name	department_name
Lex	NULL
NULL	Executive

Lex (no dept) + Executive (no employee). Neena and Alice are NOT here (they match).

CROSS JOIN

Every combination of A and B



employees = A | departments = B

CROSS JOIN returns every possible combination of rows from A and B. There is no ON condition. If A has 3 rows and B has 3 rows, the result has 3 x 3 = 9 rows. This is also called a "Cartesian product". Use it carefully because the result can be very large!

```
SELECT e.first_name, d.department_name
FROM hr.employees e
  CROSS JOIN hr.departments d;
```

Result: 9 rows (3 employees x 3 departments). Each employee is paired with each department, even if they are not related.

SELF JOIN

A table joined with itself

employees (e) = A | employees (m) = B

A SELF JOIN is when a table is joined with itself. You use the same table twice with different aliases. For example, you can find employees who earn more than their manager. The "manager_id" in the employees table points to another employee's "employee_id".

```
SELECT e.first_name AS employee,
     m.first_name AS manager
FROM hr.employees e
  LEFT JOIN hr.employees m
    ON e.manager_id = m.employee_id;
```

Quick Reference

JOIN Type	What It Returns	Rows Lost?
INNER JOIN	Only matching rows (A and B)	Yes - non-matches removed
LEFT JOIN	All from A + matching from B	B-only rows lost
RIGHT JOIN	All from B + matching from A	A-only rows lost
FULL OUTER JOIN	All from A + All from B	Nothing lost
LEFT EXCL. JOIN	A rows with NO match in B	B-only and matches lost
RIGHT EXCL. JOIN	B rows with NO match in A	A-only and matches lost
FULL EXCL. JOIN	Non-matching from both	Matching rows lost
CROSS JOIN	All combinations (A x B)	Nothing lost (but many rows!)
SELF JOIN	Table joined with itself	Depends on type (INNER/LEFT...)

Which JOIN Should You Use?

Most of the time, use LEFT JOIN. It keeps all rows from your main table (usually employees or the table in FROM). If there is no match, you get NULL instead of losing the row. This is safer because you can see what did not match and investigate.

Use INNER JOIN when you ONLY want matching rows. For example, you want a report of employees who ARE in a department. Employees without a department are not relevant for this report.

Use FULL OUTER JOIN rarely. It is useful when you need a complete picture of both tables, including rows that match and rows that do not match on either side.

Use CROSS JOIN very carefully. It creates many rows. If A has 1000 rows and B has 1000 rows, CROSS JOIN gives 1,000,000 rows! Only use it when you truly need all combinations (for example, generating a calendar or a size/color matrix).

Remember:
LEFT JOIN = "Keep everything from my main table, add info if it exists"
INNER JOIN = "Only show me rows that match on both sides"